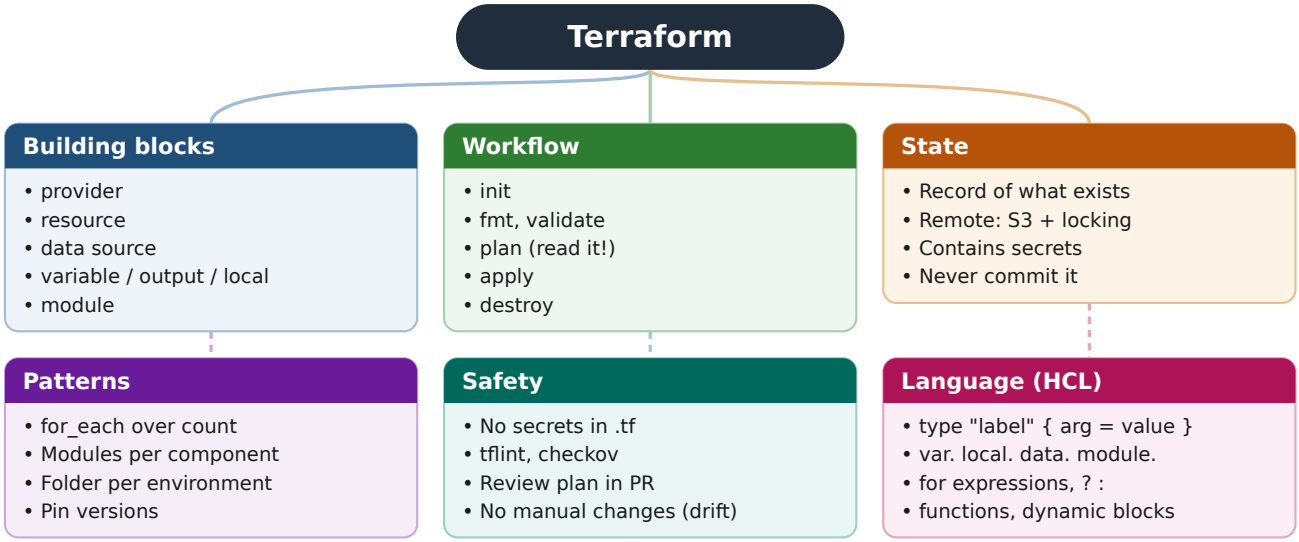
9. Terraform (Infrastructure as Code)

In this section: 9.1 Concepts · 9.2 Workflow commands · 9.3 Example · 9.4 Project layout · 9.5 Security and best practices · 9.6 Troubleshooting · 9.7 More Terraform features · 9.8 Terraform language (HCL): what is what



9.1 Concepts

Term	Meaning
Infrastructure as Code (IaC)	Infrastructure defined in code files: versioned, reviewed, repeatable
Declarative	You describe the end state; the tool works out the steps
Provider	A plugin that talks to a platform's API (aws, azurerm, kubernetes)
Resource	Something to create (`aws_instance` , `aws_s3_bucket`)
Data source	Reads existing information (latest AMI, existing VPC)
Variable / output / local	Input / printed result / internal computed value
State	`terraform.tfstate` : Terraform's record of what it created. **Contains sensitive data**
Remote state	State stored in S3 (with locking) so a team shares it safely
Module	A reusable folder of Terraform code
Plan	Preview of changes: `+` create, `~` change, `-` destroy, `-/+` replace

9.2 Workflow commands

Command	What it does
`terraform init`	Download providers/modules, set up the backend

Command	What it does
`terraform fmt`	Format the code
`terraform validate`	Check syntax
`terraform plan`	Show what will change. **Always read it**
`terraform apply`	Make the changes (asks for confirmation)
`terraform destroy`	Delete everything in this configuration
`terraform output`	Show outputs
`terraform state list` / `state show <addr>`	Inspect state
`terraform import <addr> <id>`	Bring an existing resource under Terraform
`terraform plan -out=tfplan` → `apply tfplan`	Apply exactly what was reviewed

9.3 Example

```
# versions.tf
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    bucket       = "my-tf-state-bucket"
    key          = "dev/network/terraform.tfstate"
    region       = "ap-south-1"
    use_lockfile = true # state locking with S3
    encrypt      = true
  }
}

provider "aws" {
  region = var.region
  default_tags { tags = { Project = "devops-lab", Env = var.env } }
}

# variables.tf
variable "region" {
  type    = string
  default = "ap-south-1"
}

variable "env" {
  type = string
}

# main.tf
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id # reference creates a dependency
  cidr_block              = "10.0.1.0/24"
  map_public_ip_on_launch = true
}

# outputs.tf
output "vpc_id" { value = aws_vpc.main.id }
```

```
# terraform.tfvars  (values per environment, no secrets)
env = "dev"
```

Module use:

```
module "network" {
  source     = "./modules/network"
  cidr_block = "10.0.0.0/16"
}
```

Other useful features: `count`, `for_each` (create many), `depends_on`, `lifecycle { prevent_destroy = true }`, `sensitive = true` on variables/outputs.

9.4 Project layout

```
infra/
├── modules/
│   └── network/ (main.tf, variables.tf, outputs.tf)
└── envs/
    ├── dev/  (main.tf, backend.tf, terraform.tfvars)
    └── prod/
```

9.5 Security and best practices

- Remote state in an encrypted S3 bucket with locking and versioning. Never commit `*.tfstate` or `.terraform/`.
- No secrets in `.tf` or `.tfvars` files. Read them from Secrets Manager/SSM or environment variables (`TF_VAR_name`).
- Pin provider and module versions. Commit `.terraform.lock.hcl`.
- Review `plan` in pull requests. Run `fmt`, `validate`, `tflint`, and a security scanner (`checkov` / `trivy config`) in CI.
- Small, separate states per environment/component. Least-privilege IAM role for Terraform.
- Don't change Terraform-managed resources by hand (causes **drift**).

9.6 Troubleshooting

Error	Fix
State lock error	Someone else is running it, or a crashed run left a lock. Wait, or `terraform force-unlock <id>` only if sure
Access denied / unauthorized	Check `aws sts get-caller-identity`, profile, region, IAM permissions
Resource already exists	Import it, or change the name
Plan wants to replace something unexpectedly	Read which attribute forces replacement; fix the code or use `lifecycle`
Dependency/order errors	Use references instead of hard-coded IDs; `depends_on` if needed
Drift	`terraform plan` shows it; decide whether code or the real resource is correct

9.7 More Terraform features

```
# Data source: look up the latest Ubuntu AMI instead of hard-coding an ID
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }
}

# for_each: one resource per item in a map (stable names, safe to add/remove)
variable "subnets" {
  default = {
    public-a = "10.0.1.0/24"
    public-b = "10.0.2.0/24"
  }
}

resource "aws_subnet" "this" {
  for_each   = var.subnets
  vpc_id     = aws_vpc.main.id
  cidr_block = each.value
  tags       = { Name = each.key }
}
```

Command / feature	Purpose
`terraform apply -replace=aws_instance.web`	Force one resource to be recreated (replaces old `taint`)
`terraform plan -refresh-only`	Detect drift without changing anything
`terraform console`	Try expressions interactively
`terraform workspace new dev`	Separate state per workspace (simple setups; many teams prefer one folder per environment)
`moved { from = ... to = ... }`	Rename/move resources in code without destroying them
`count` vs `for_each`	`count` for N identical copies; `for_each` for named items (preferred)

OpenTofu is an open-source fork of Terraform with almost the same commands.

9.8 Terraform language (HCL): what is what

Anatomy of a block

```
# block type = resource | label 1 = resource type | label 2 = your name for it
resource "aws_instance" "web" {
  ami           = "ami-0abc123"     # ARGUMENT: name = value
  instance_type = var.instance_type # a value can reference other things
  tags = {                          # map value
    Name = "web-1"
  }
  root_block_device { # NESTED BLOCK (no "=")
    volume_size = 20
  }
}
```

- **Block** = `type "labels" { body }`. **Argument** = `name = value`. **Nested block** = `name { ... }` without `=`.
- The resource's **address** is `aws_instance.web`. Other code reads its attributes, e.g. `aws_instance.web.public_ip`.
- Terraform reads **all `.tf` files in the folder** together; file names are only for humans.

Block types

Block	Purpose	Example
`terraform`	Terraform version, required providers, backend (state location)	`terraform { required_version = ">= 1.6" }`
`provider`	Configures a platform's API (region, credentials from environment/role)	`provider "aws" { region = "ap-south-1" }`
`resource`	Something Terraform **creates and manages**	`resource "aws_s3_bucket" "logs" { ... }`
`data`	**Reads** something that already exists	`data "aws_availability_zones" "available" {}`
`variable`	An **input**	`variable "env" { type = string }`
`locals`	**Computed helper values** inside the module	`locals { name = "${var.env}-web" }`
`output`	A value printed after apply / used by other modules	`output "ip" { value = aws_instance.web.public_ip }`
`module`	Uses a reusable folder of Terraform code	`module "vpc" { source = "./modules/vpc" }`
`moved` / `import`	Refactor without destroying / adopt existing resources	`import` block with `to = aws_s3_bucket.logs` and `id = "my-bucket"`

How to reference things

You write	It means
`var.env`	Input variable `env`
`local.name`	Local value `name`
`aws_vpc.main.id`	Attribute `id` of resource `aws_vpc` named `main`
`data.aws_ami.ubuntu.id`	Attribute of a data source
`module.vpc.vpc_id`	Output `vpc_id` of module `vpc`
`aws_subnet.private[0].id`	Item 0 when the resource uses `count`
`aws_subnet.this["public-a"].id`	Item by key when the resource uses `for_each`
`each.key` / `each.value`	Current item inside a `for_each` resource
`count.index`	Current number (0, 1, 2…) inside a `count` resource
`path.module`	Folder of the current module (for files/templates)

Implicit dependencies: referencing `aws_vpc.main.id` inside a subnet tells Terraform to create the VPC first. Use `depends_on` only when there is no reference.

Data types

Type	Example
`string`	`"t3.micro"`
`number`	`3`
`bool`	`true`
`list(string)`	`["a", "b"]` — ordered, accessed by index `[0]`
`set(string)`	`toset(["a", "b"])` — unique, no order
`map(string)`	`{ env = "dev", team = "ops" }` — accessed by key `["env"]`
`object({...})`	`{ name = string, size = number }` — fixed structure

Variables in detail

```
variable "instance_type" {
  description = "EC2 size for the web tier" # documents the input
  type        = string                      # type constraint
  default     = "t3.micro"                  # optional: no default = required input

  validation { # reject bad values early
    condition     = contains(["t3.micro", "t3.small"], var.instance_type)
    error_message = "Use t3.micro or t3.small."
  }
}

variable "db_password" {
  type      = string
  sensitive = true # hidden in plan/apply output (still stored in state!)
}
```

Ways to set variables (later ones win): `default` → `terraform.tfvars` / `*.auto.tfvars` → `-var-file=prod.tfvars` → `-var="env=prod"`. Environment variable form: `TF_VAR_db_password`.

Expressions

```
locals {
  name        = "${var.project}-${var.env}"                    # string interpolation
  is_prod     = var.env == "prod"                              # comparison → bool
  size        = local.is_prod ? "t3.small" : "t3.micro"        # conditional: cond ? a : b
  upper_names = [for s in var.servers : upper(s)]              # for expression → list
  name_map    = { for s in var.servers : s => "${s}.internal" } # for expression → map
  subnet_ids  = aws_subnet.private[*].id                       # splat: every id
}
```

Useful built-in functions

Function	Example	Result
`length`	`length(["a","b"])`	`2`
`upper` / `lower`	`upper("dev")`	`"DEV"`
`join` / `split`	`join(",", ["a","b"])`	`"a,b"`
`merge`	`merge(local.tags, { Name = "web" })`	Combined map
`concat`	`concat(["a"], ["b"])`	`["a","b"]`

Function	Example	Result
`lookup`	`lookup(var.sizes, "prod", "t3.micro")`	Value or default
`contains`	`contains(["a","b"], "a")`	`true`
`cidrsubnet`	`cidrsubnet("10.0.0.0/16", 8, 1)`	`"10.0.1.0/24"`
`file` / `templatefile`	`templatefile("${path.module}/init.sh.tftpl", { port = 80 })`	File content (with variables)
`try` / `coalesce`	`try(var.cfg.size, "small")`	Fallback when missing
`toset` / `tolist`	`toset(var.names)`	Type conversion

Try any function in `terraform console`.

Meta-arguments (work on every resource)

Meta-argument	Purpose
`count = 3`	Create N copies (`count.index`)
`for_each = toset([...])` or a map	One copy per item (`each.key` , `each.value`) — preferred
`depends_on = [aws_iam_role.x]`	Force an order when there is no reference
`provider = aws.us_east`	Use an aliased provider (e.g. second region)
`lifecycle { ... }`	`create_before_destroy` , `prevent_destroy` , `ignore_changes`

Dynamic block (repeat a nested block from a list):

```
variable "ingress_ports" {
  type    = list(number)
  default = [80, 443]
}

resource "aws_security_group" "web" {
  name   = "web-sg"
  vpc_id = aws_vpc.main.id

  dynamic "ingress" {
    for_each = var.ingress_ports
    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

Complete annotated example: VPC + public subnet + web server

```
# ---------- versions.tf ----------
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws" # where to download the provider
      version = "~> 5.0"        # any 5.x version
    }
  }
}

provider "aws" {
  region = var.region # credentials come from your AWS CLI profile / IAM role
  default_tags {
    tags = local.common_tags # added to every resource automatically
  }
}

# ---------- variables.tf ----------
variable "region" {
  type    = string
  default = "ap-south-1"
}

variable "env" {
  type        = string
  description = "Environment name, e.g. dev or prod"
}

variable "my_ip_cidr" {
  type        = string
  description = "Your public IP for SSH, e.g. 203.0.113.10/32"
}

# ---------- main.tf ----------
locals {
  name        = "lab-${var.env}"
  common_tags = { Project = "devops-lab", Env = var.env, ManagedBy = "terraform" }
}

data "aws_availability_zones" "available" {
  state = "available" # read the AZ names of this region
}

data "aws_ami" "ubuntu" { # find the newest Ubuntu 24.04 image
  most_recent = true
  owners      = ["099720109477"] # Canonical's account
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }
}

resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags                 = { Name = "${local.name}-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id # reference → VPC is created first
}

resource "aws_subnet" "public" {
```

```
  vpc_id                  = aws_vpc.main.id
  cidr_block              = cidrsubnet(aws_vpc.main.cidr_block, 8, 1) # 10.0.1.0/24
  availability_zone       = data.aws_availability_zones.available.names[0]
  map_public_ip_on_launch = true
  tags                    = { Name = "${local.name}-public-a" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"                 # everything else ...
    gateway_id = aws_internet_gateway.igw.id # ... goes to the internet gateway
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

resource "aws_security_group" "web" {
  name   = "${local.name}-web"
  vpc_id = aws_vpc.main.id

  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    description = "SSH only from my IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip_cidr]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1" # all protocols
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "web" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t3.micro"
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web.id]
  user_data              = <<-EOT
    #!/bin/bash
    apt-get update -y
    apt-get install -y nginx
    systemctl enable --now nginx
  EOT
  tags                   = { Name = "${local.name}-web" }
}

# ---------- outputs.tf ----------
output "web_url" {
  description = "Open this in a browser"
  value       = "http://${aws_instance.web.public_ip}"
}
```

Run it: `terraform init` → `terraform plan -var="env=dev" -var="my_ip_cidr=<your-ip>/32"` → `terraform apply ...` → open the URL → **`terraform destroy ...`** when finished.

Naming conventions: snake_case names (`aws_instance.web_server`), one purpose per file (`versions.tf`, `variables.tf`, `main.tf`, `outputs.tf`), `this` as the name when a module has only one resource of a type, run `terraform fmt` before every commit.

> **REMEMBER**
>
> - The loop: **write → init → plan → apply**.
> - Plan symbols: **+** create · **~** update · **-** destroy · **-/+** replace (check why!).